# IC526 Advanced Compilers
# Lec 15:
# Refinement and Alive2

Yoonseung Kim
DGIST EECS
Fall 2026

# Review: Programs and Behaviors

**A Normal (Safe) Program**

```
fn main() -> i32 {
  let a: [i32; 2] = [7, 7];
  println!("{}", a[1]);
  return 0;
}
```

```
Beh(P) =
  { print(7) · return(0) }
```

**An Unsafe Program**

```
fn main() -> i32 {
  let a: [i32; 2] = [7, 7];
  println!("{}", a[2]);
  return 0;
}
```

```
Beh(P) = All_Traces
```

In our RustY model, an out-of-bounds access is UB.

# Refinement

# What Do We Expect from a Compiler?

Beh(P): all complete observable traces permitted by P.

**Equivalence** $Beh(T) = Beh(S)$

✓ **Refinement** $Beh(T) \subseteq Beh(S)$

- The target may permit fewer traces.
- Every target trace must be permitted by the source.

# Why Refinement #1: Resolving Nondeterminism

Some languages (e.g., C) leave argument evaluation order unspecified.

```
int mark(int x) {
  printf("%d", x);
  return x;
}

int add(int x, int y) {
  return x + y;
}

int main() {
  return add(mark(1), mark(2));
}
```

**Source permits both traces**

```
print(1) · print(2) · return(3)
print(2) · print(1) · return(3)
```

```
...


define i32 @main() {
  %v1 = call i32 @mark(i32 1)
  %v2 = call i32 @mark(i32 2)
  %ret = add i32 %v1, %v2
  ret i32 %ret
}
```

**Target evaluates left to right**

```
print(1) · print(2) · return(3)
```

# Why Refinement #2: Aggressive Optimizations

**RustY**

```
fn f(i: usize) -> i32 {
  let a: [i32; 2] =
      [7, 7];
  return a[i];
}
```

**Generated LLVM IR**

```
define i32 @f(i64 %arg) {
  %a = alloca [2 x i32]
  %ip = alloca i64
  store i64 %arg, ptr %ip
  store [2 x i32]
    [i32 7, i32 7], ptr %a
  %i = load i64, ptr %ip
  %p = getelementptr [2 x i32],
         ptr %a, i64 0, i64 %i
  %v = load i32, ptr %p
  ret i32 %v
}
```

**Optimized LLVM IR**

```
define i32 @f(
    i64 %arg) {
  ret i32 7
}
```

Refinement justifies aggressive optimizations **using UB**.

# Caveat: Randomness Is an External Interaction

Consider a call to an external random API.

```
Source: x = random(); print(x); return 0;
Target: x = random(); print(0); return 0;
```

Record the API response in the trace. Suppose it returns 1.

```
Source: random() ↦ 1 · print(1) · return(0)
Target: random() ↦ 1 · print(0) · return(0)
```

➔ For the same response, the target prints a value the source forbids.

# Caveat: Can Refinement Remove Inputs?

For simplicity, read() returns either 1 or 2.

**Source S**

```
x = read();
print(x + 1);
return 0;
```

```
Beh(S) = {
  read() ↦ 1 · print(2) · return(0),
  read() ↦ 2 · print(3) · return(0)
}
```

**Proposed behavior set for T?**

Keep the execution for input 1.

Delete every execution for input 2.

```
Beh(T?) = {
  read() ↦ 1 · print(2) · return(0)
}
```

Beh(T?) ⊆ Beh(S) holds.

Can any program using read() have exactly this target behavior set?

# Example 1: Removing an Addition

**Source**

```
%a = add i32 %x, 0
ret i32 %a
```

**Target**

```
ret i32 %x
```

Why does the target refine the source?

# Example 2: Reusing a Computed Value

**Source**

```
%a = add i32 %x, %y
%b = add i32 %x, %y
%r = mul i32 %a, %b
ret i32 %r
```

**Target**

```
%a = add i32 %x, %y

%r = mul i32 %a, %a
ret i32 %r
```

Why does the target refine the source?

# Example 3: Replacing a Private Slot

**RustY**

```
fn f(x: i32) -> i32 {
    let mut y: i32 = x;
    y = y + 1;
    return y;
}
```

**Source**

```
%p = alloca i32
store i32 %x, ptr %p
%a = load i32, ptr %p
%b = add i32 %a, 1
store i32 %b, ptr %p
%r = load i32, ptr %p
ret i32 %r
```

**Target**

```
%b = add i32 %x, 1

ret i32 %b
```

Why does the target refine the source?

# Example 4: Making a Value More Defined

**Source**

```
%r = mul i32 %x, 0
ret i32 %r
```

**Target**

```
ret i32 0
```

| Input %x | Source result | Target result | Relation before return |
|---|---|---|---|
| Ordinary integer | `0` | `0` | `0 ≥ 0` |
| poison | `poison` | `0` | `poison ≥ 0` |

# Example 5: Incorrect Optimizations

## Guarded array access

in_bounds(i) means $0 \le i < n$.

```
S: if in_bounds(i):
     return a[i] + a[i]
   else:
     return 0

T: v = a[i]
   if in_bounds(i):
     return v + v
   else:
     return 0
```

## Loop-invariant division

Assume y does not change in the loop.

```
S: while c:
     c = f(10 / y)
   return 0

T: t = 10 / y
   while c:
     c = f(t)
   return 0
```

What happens if i = n, or if initially c = false and y = 0?

# Alive2:
# Automated Refinement Checking

# Alive2: LLVM IR Refinement Checking

Alive2 uses SMT solving to check LLVM IR transformations.

`https://alive2.llvm.org/ce/`

1. Click inside the LLVM IR source editor on the left.
2. Define functions named @src and @tgt in the same editor.
3. Choose alive-tv and leave “Compiler options” empty.
4. Read the automatically updated output on the right.

# Experiment 1: Addition by Zero

Recall: every target trace must be permitted by the source.

```
define i8 @src(i8 %x) {
  %r = add i8 %x, 0
  ret i8 %r
}

define i8 @tgt(i8 %x) {
  ret i8 %x
}
```

➔ Transformation seems to be correct!

# Experiment 2: Adding nsw

Can we add nsw without ruling out signed overflow?

```
define i8 @src(i8 %x) {
  %r = add i8 %x, 1
  ret i8 %r
}

define i8 @tgt(i8 %x) {
  %r = add nsw i8 %x, 1
  ret i8 %r
}
```

➔ Transformation doesn't verify!

# Reading an Overflow Counterexample

```
Example:
i8 %x = #x7f (127)
```

```
Source:
i8 %r = #x80 (128, -128)
```

The source guarantees
a concrete result on this input.

```
Target:
i8 %r = poison
Source value: #x80 (128, -128)
Target value: poison
```

The target introduces poison,
so refinement fails.

# Reverse Direction: Removing nsw

```
define i8 @src(i8 %x) {
  %r = add nsw i8 %x, 1
  ret i8 %r
}

define i8 @tgt(i8 %x) {
  %r = add i8 %x, 1
  ret i8 %r
}
```

➔ Transformation seems to be correct!

# Experiment 3: GVN and Repeated Loads

Can we reuse the first load's result?

```
define i32 @src(ptr %p) {
  %a = load i32, ptr %p
  %b = load i32, ptr %p
  %r = add i32 %a, %b
  ret i32 %r
}

define i32 @tgt(ptr %p) {
  %a = load i32, ptr %p
  %r = add i32 %a, %a
  ret i32 %r
}
```

➔ Transformation seems to be correct!

# GVN: An Intervening Store Matters

```
define i32 @src(ptr %p, ptr %q) {
  store i32 0, ptr %p
  %a = load i32, ptr %p
  store i32 7, ptr %q
  %b = load i32, ptr %p
  ret i32 %b
}

define i32 @tgt(ptr %p, ptr %q) {
  store i32 0, ptr %p
  %a = load i32, ptr %p
  store i32 7, ptr %q
  ret i32 %a
}
```

➔ Transformation doesn't verify!

# GVN Counterexample: Aliasing Pointers

```
define i32 @src(ptr %p, ptr %q) {
  store i32 0, ptr %p
  %a = load i32, ptr %p
  store i32 7, ptr %q
  %b = load i32, ptr %p
  ret i32 %b
}

define i32 @tgt(ptr %p, ptr %q) {
  store i32 0, ptr %p
  %a = load i32, ptr %p
  store i32 7, ptr %q
  ret i32 %a
}
```

**Alive2 output (excerpt)**

```
Example:
ptr %p = pointer(non-local,
  block_id=1, offset=0)
  / Address=#x04
ptr %q = pointer(non-local,
  block_id=1, offset=0)
  / Address=#x04

Source value: #x00000007 (7)
Target value: #x00000000 (0)
```

**p = q**

# Experiment 4: LICM and a Safe Move

This loop runs either zero or two iterations.

```
define i32 @src(i1 %go, i32 %x) {
entry:
  br i1 %go, label %loop, label %exit
loop:
  %i = phi i32 [0,%entry], [%j,%loop]
  %v = add i32 %x, 1
  %j = add i32 %i, 1
  %c = icmp ult i32 %j, 2
  br i1 %c, label %loop, label %exit
exit:
  %r = phi i32 [0,%entry], [%v,%loop]
  ret i32 %r
}
```

```
define i32 @tgt(i1 %go, i32 %x) {
entry:
  %v = add i32 %x, 1
  br i1 %go, label %loop, label %exit
loop:
  %i = phi i32 [0,%entry], [%j,%loop]
  %j = add i32 %i, 1
  %c = icmp ult i32 %j, 2
  br i1 %c, label %loop, label %exit
exit:
  %r = phi i32 [0,%entry], [%v,%loop]
  ret i32 %r
}
```

➔ Transformation seems to be correct!

# LICM: A Loop May Execute Zero Times

Hoisting introduces UB when go = false and x = 0.

```
define i32 @src(i1 %go, i32 %x) {
entry:
  br i1 %go, label %loop, label %exit
loop:
  %i = phi i32 [0,%entry], [%j,%loop]
  %v = sdiv i32 10, %x
  %j = add i32 %i, 1
  %c = icmp ult i32 %j, 2
  br i1 %c, label %loop, label %exit
exit:
  %r = phi i32 [0,%entry], [%v,%loop]
  ret i32 %r
}
```

```
define i32 @tgt(i1 %go, i32 %x) {
entry:
  %v = sdiv i32 10, %x
  br i1 %go, label %loop, label %exit
loop:
  %i = phi i32 [0,%entry], [%j,%loop]
  %j = add i32 %i, 1
  %c = icmp ult i32 %j, 2
  br i1 %c, label %loop, label %exit
exit:
  %r = phi i32 [0,%entry], [%v,%loop]
  ret i32 %r
}
```

➔ Transformation doesn't verify!

# A Limit: Bounded Loop Unrolling

Both loops execute five iterations.

```
define i32 @src() {
entry:
  br label %loop
loop:
  %i = phi i32 [0,%entry], [%j,%loop]
  %j = add i32 %i, 1
  %c = icmp ult i32 %j, 5
  br i1 %c, label %loop, label %exit
exit:
  ret i32 0
}
```

```
define i32 @tgt() {
entry:
  br label %loop
loop:
  %i = phi i32 [0,%entry], [%j,%loop]
  %j = add i32 %i, 1
  %c = icmp ult i32 %j, 5
  br i1 %c, label %loop, label %exit
exit:
  ret i32 1
}
```

ERROR: The source program doesn't reach a return instruction.

Default result: inconclusive. Increase the unrolling bound.

```
Compiler options: --src-unroll=6 --tgt-unroll=6
```